

AI-Powered Code Debugging Assistant using CrewAI

Abstract

This project presents a multi-agent AI-powered code debugging assistant built using CrewAI. The system combines three collaborating agents, structured Pydantic domain models, a Builder Pattern, CodeInterpreterTool integration, and an interactive Streamlit dashboard to automate Python code analysis and correction. Unlike prompt-only approaches, the proposed architecture separates LLM reasoning from artifact construction, producing deterministic, strongly typed workflow artifacts. This design improves robustness, maintainability, and observability while providing a user-friendly debugging experience.

1. Introduction

Large Language Models have significantly improved automated programming assistance. However, directly requesting complex structured outputs from LLMs often leads to inconsistent JSON, schema validation failures, and unreliable downstream processing. The objective of this project was to design a dependable AI-assisted debugging workflow using CrewAI while maintaining clean software engineering practices. The solution employs three specialized agents that collaborate sequentially to analyze, correct, and verify Python programs.

2. Objectives

- Build a sequential CrewAI workflow.
- Analyze Python source code using an Analyzer Agent.
- Execute code using CodeInterpreterTool when appropriate.
- Generate corrected code using a dedicated Corrector Agent.
- Coordinate execution through a Manager Agent.
- Convert lightweight LLM outputs into strongly typed domain artifacts.
- Visualize results through an interactive Streamlit dashboard.

3. System Architecture

The system consists of a Streamlit front-end, a Debugging Service, a sequential CrewAI workflow, Builder components, structured Pydantic artifacts, and an observability layer. The user submits Python source code through the dashboard. The Debugging Service invokes the CrewAI workflow where the Analyzer first examines the program and optionally executes it using the CodeInterpreterTool. The Corrector receives both the

original source code and the analysis results before generating a corrected implementation. The Manager supervises the overall workflow and verifies successful completion.

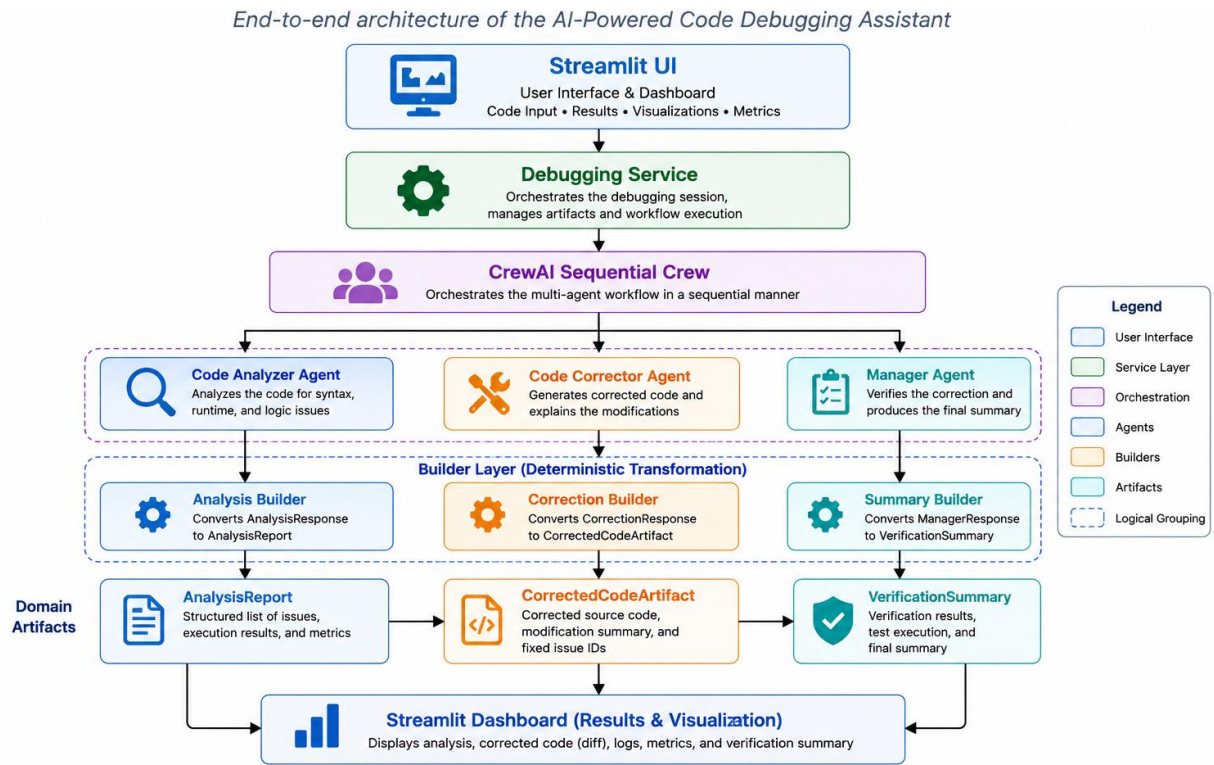


Figure 1 – Overall System Architecture

4. Multi-Agent Workflow

The workflow follows a strict sequential execution model.

Analyzer Agent:

Responsible for identifying syntax, runtime, and logical issues. The analyzer returns a lightweight AnalysisResponse rather than directly constructing complex artifacts.

Corrector Agent:

Receives the source code and structured analysis, applies minimal targeted modifications, and produces corrected code.

Manager Agent:

Coordinates execution, validates task completion, and summarizes the debugging session.

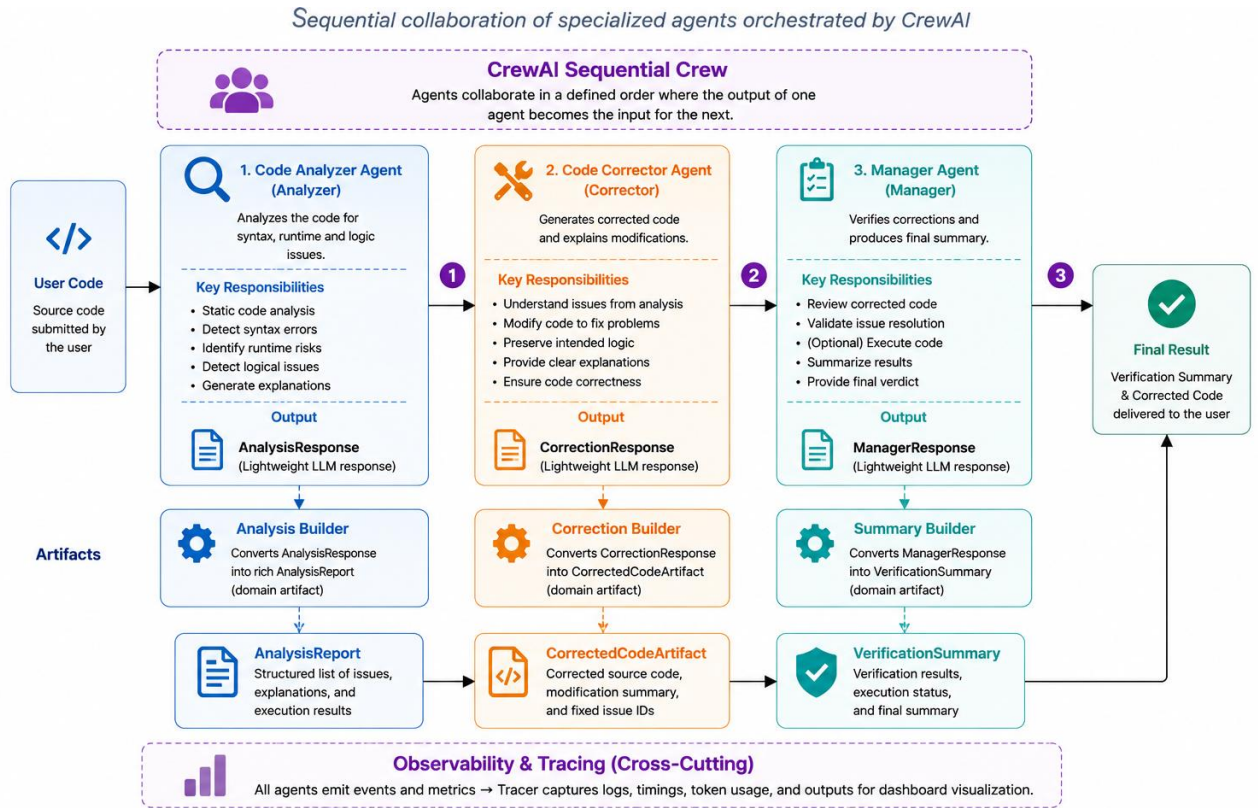


Figure 2 – Multi-Agent Workflow

5. Builder Pattern

A key contribution of this project is the Builder Pattern. Earlier versions relied on LLMs to generate complete Pydantic objects, resulting in validation failures due to missing UUIDs, timestamps, and enum mismatches. Version 1.0 introduces lightweight response schemas that contain only reasoning outputs. Dedicated Builder classes then construct complete domain artifacts by generating identifiers, timestamps, metadata, computed fields, and strongly typed objects.

This separation offers several advantages:

- deterministic artifact creation
- reduced prompt complexity
- fewer schema validation failures
- easier maintenance
- cleaner software architecture

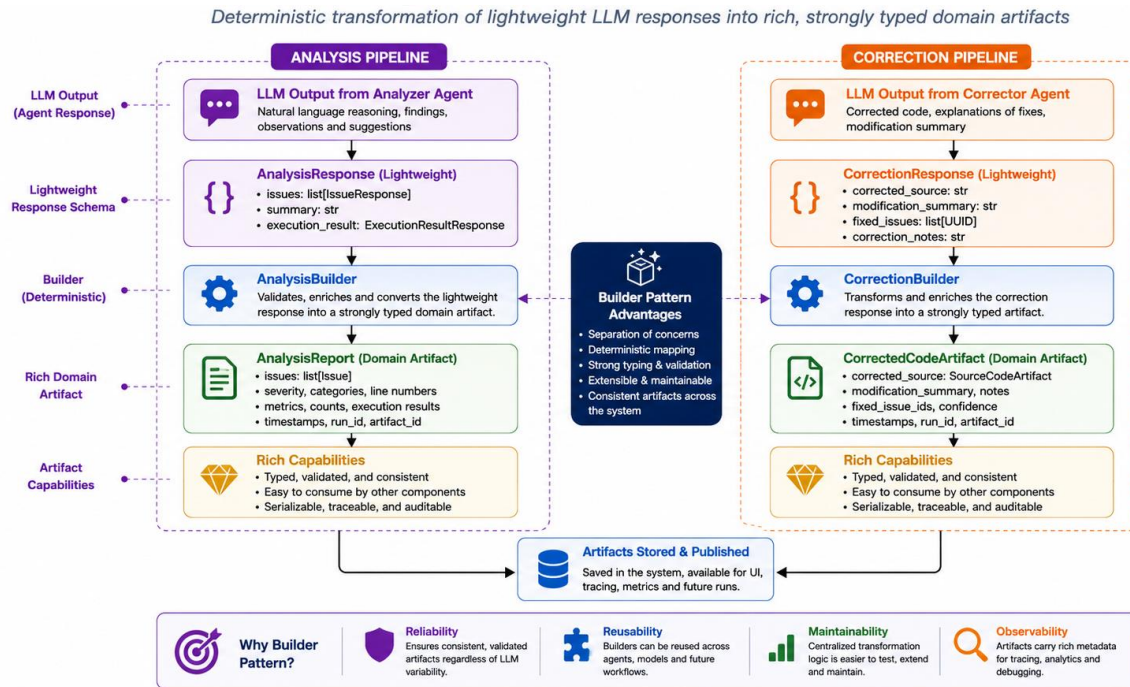


Figure 3 – Builder Pattern Architecture

6. Domain Artifact Lifecycle

The debugging workflow exchanges structured artifacts throughout the pipeline. SourceCodeArtifact represents the original user program. AnalysisBuilder converts AnalysisResponse into AnalysisReport containing detected issues, execution results, and summary statistics. CorrectionBuilder transforms CorrectionResponse into CorrectedCodeArtifact while preserving workflow metadata. This artifact-centric design improves traceability and simplifies future extensions.

Lifecycle of artifacts produced and consumed across the debugging workflow



Figure 4 – Domain Artifact Flow

7. Observability

To improve transparency, the application includes a lightweight observability framework. A custom Tracer records workflow execution events, task completion, execution outcomes, and runtime metrics. These records are exported as JSON traces and visualized within the Streamlit interface. The observability layer enables easier debugging of both agent behavior and application execution.

End-to-end observability from agent execution to insights and dashboards

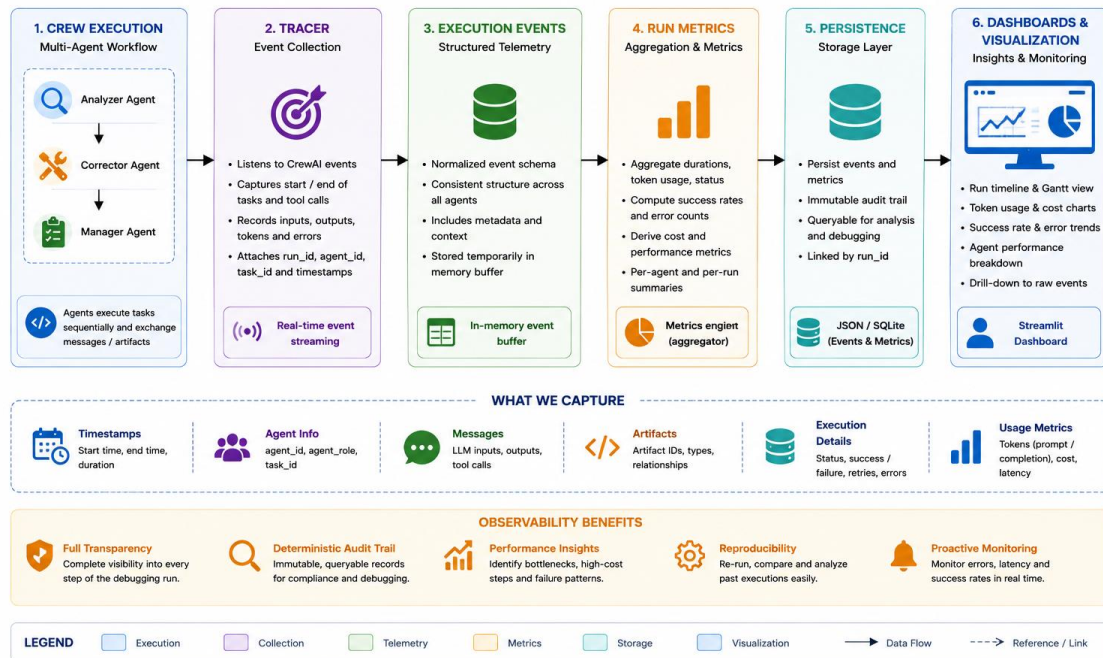


Figure 5 – Observability Pipeline

8. Streamlit Application

The Streamlit interface provides an end-to-end user experience. Users can paste Python code, execute the debugging workflow, inspect identified issues, compare original and corrected code, review execution results, visualize workflow progress, and inspect runtime metrics. Side-by-side code comparison and unified diff views make changes easy to understand.

localhost:8501

Deploy

Code debugging console

analyzer → corrector → manager

PYTHON SOURCE

```
1 def generate(self, numRows: int) -> List[List[int]]:
2     if numRows == 1:
3         return [[1]]
4     if numRows == 2:
5         return [[1], [1, 1]]
6     triangle = [[1], [1, 2]]
7     rows = 3
8     while rows <= numRows:
9         current_row = []
10        for i in range(len(triangle[rows-1])):
11            current_row.append(triangle[rows-1][i] + triangle[rows-1][i-1])
12        current_row = [1] + current_row
13        triangle.append(current_row)
14        rows += 1
15
16    return triangle
```

Analyze

✓ Analyzer · 12685 ms

✓ Corrector · 1696 ms

✓ Manager verification · 22380 ms

Each checkmark reflects one completed CrewAI task — not live progress.

Run status: Completed

Agents: 3

Issues found: 1

Issues fixed: 1

Completed in 43.6s

Analysis | Corrected code | Verification | Observability

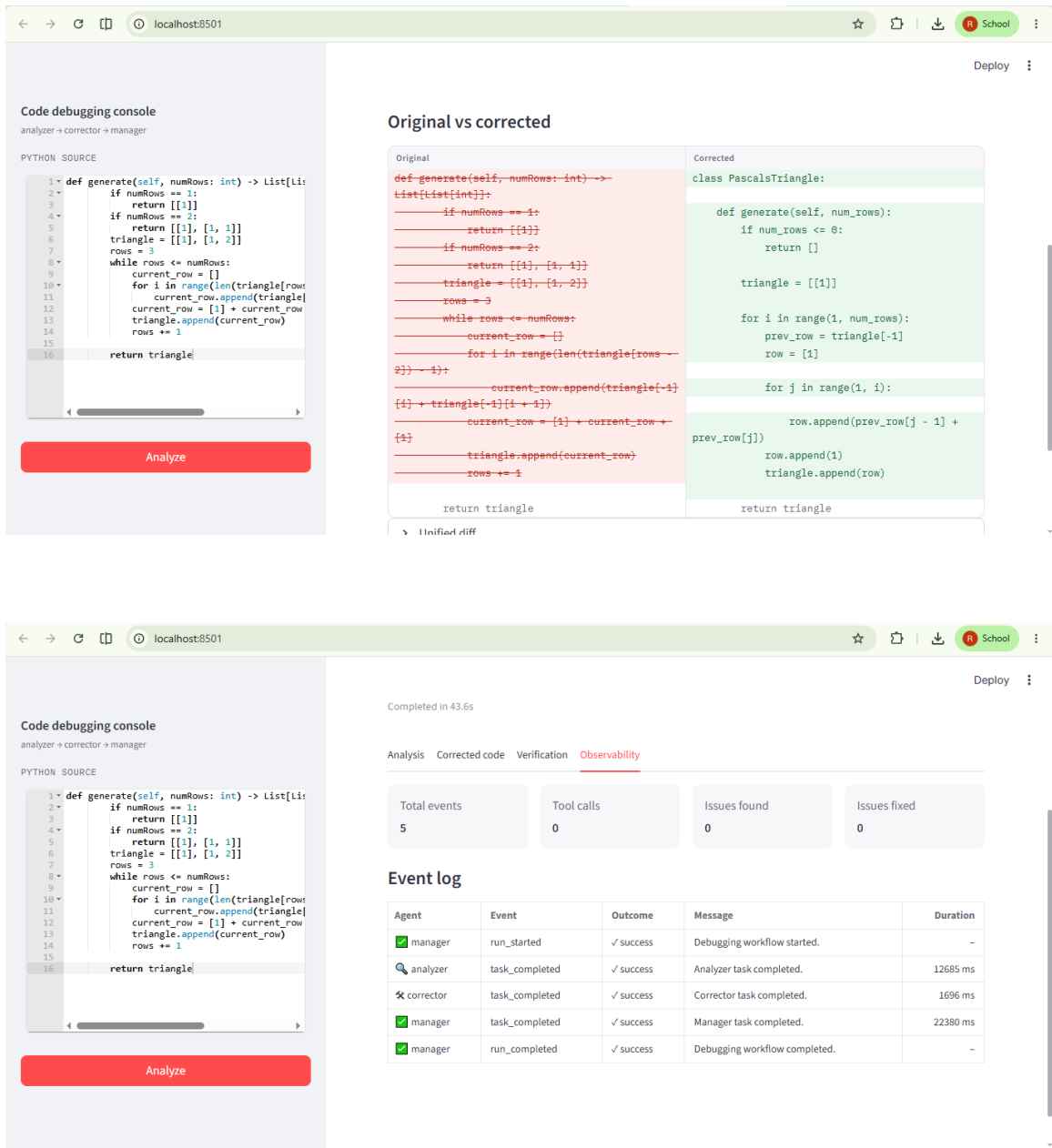
Summary: The function fails to generate the correct sequence for Pascal's Triangle because it starts with an incorrect hardcoded list `[[1, 2]]` for the second row instead of `[1, 1]`. Consequently, subsequent calculations propagate this error, leading to an incorrect result (e.g., `[1, 5, 8, 5, 1]` instead of `[1, 4, 6, 4, 1]` for the 4th row).

Execution result

Execution: ran · success

stdout

```
[[1], [1, 2], [1, 3, 1], [1, 4, 4, 1], [1, 5, 8, 5, 1]]
```



Application Screenshots

9. Technology Stack

Framework: CrewAI

Programming Language: Python

UI: Streamlit

Validation: Pydantic v2

Execution Tool: CodeInterpreterTool

LLM: Groq Llama 3.3 70B

Observability: Custom Tracer

10. Results and Discussion

The completed system successfully demonstrates multi-agent collaboration for automated code debugging. The Builder Pattern significantly improved workflow stability by eliminating many schema validation problems encountered during early development. The resulting architecture is modular, extensible, and suitable for future enhancements such as benchmark-driven evaluation and support for additional programming languages.

11. Limitations and Future Work

Version 1.0 focuses on architectural robustness rather than exhaustive evaluation. Future work includes an automated evaluation benchmark suite, prompt optimization, regression testing, verification artifacts, support for additional languages, and improved logical bug detection.

12. Conclusion

This project demonstrates a practical architecture for AI-assisted code debugging using CrewAI. By separating LLM reasoning from domain artifact construction through the Builder Pattern, the system achieves greater reliability and maintainability while preserving the flexibility of LLM-based reasoning. The integration of observability, structured artifacts, and an interactive dashboard results in a professional end-to-end debugging assistant suitable for educational demonstrations and future research.